Department of
Computer Science and Engineering

Title: AWT Event Handling (ActionListener)

Object Oriented Programming Lab
CSE 202

Green University of Bangladesh

1 Objective(s)

- Understanding ActionListener and ActionEvent and using them to add events

2 Problem Statement

We can use awt in java too add events. There are many kinds of events that we can add. We will focus on ActionListener here. But apart from that are there are other kinds of events too such as MouseListener, MouseMotionListener, ItemListener, WindowListener etc.

- We will add events to swing based programs
- We will create a window where we take 2 integers and divide them. We calculate the division when a button division is clicked. We also use swing to send messages to the user while handling exception.

3 Terms

3.1 ActionListener

Java ActionListener is an interface. It gets notified whenever user click on some button and performs some form of work. In Java ActionListener gets notified through ActionEvent. The ActionListener interface is found in java.awt.event package. It has only one method: actionPerformed(). While using ActionListener one needs to override this method.

3.2 actionPerformed()

actionPerformed() is the only method under ActionListener. As Java ActionListener is an interface and actionPerformed() is provided, we need to override this method during writing the event. We register components such as buttons and when the button is used actionPerformed() gets invoked.

```
public abstract void actionPerformed(ActionEvent e);
```

3.3 Steps to use ActionListener

- Implement the ActionListener class

```
public class Lab implements ActionListener
```

- Registering the component to ActionListener

```
component.addActionListener(instanceOfListenerclass);
```

- Then override the actionPerformed() method

```
public void actionPerformed(ActionEvent e){
            //write code here
}
```

4 Implementation

- as before create a java project
- create a class extending JFrame and implementing ActionListener interface
- create necessary buttons, textfields and add them to container
- take 2 integers using textfield
- override actionPerformed() and write code for division
- add exception handling according to requirement

© Dept. of Computer Science and Engineering, GUB

4.1 Code:

```java

package lab;


import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.AWTEventListener;
import java.awt.event.ActionListener;

public class Lab extends JFrame implements ActionListener{

    JButton jb1;
    JTextField jt1, jt2;
    JLabel lbl;

    Lab6(){
        jt1=new JTextField();
        jt1.setBounds(100, 50, 150, 40);
        add(jt1);

        jt2=new JTextField();
        jt2.setBounds(100,110,150,40);
        add(jt2);

        lbl=new JLabel("Result");
        lbl.setBounds(20, 300, 150, 40);
        add(lbl);

        jb1=new JButton("Division");
        jb1.setBounds(90,220,150,30);
        add(jb1);

        jb1.addActionListener(this);

        setLayout(null);
        setSize(500,500);
        setVisible(true);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void actionPerformed(ActionEvent e){

            int a=0,b=0;

        try{
            a=Integer.parseInt(jt1.getText());
            b=Integer.parseInt(jt2.getText());
        }
        catch(NumberFormatException ne){
            JOptionPane.showMessageDialog(this,"Enter integers");
        }

        double c=0;

        if(e.getSource().equals(jb1)){

```

```
            try{
                c=a/b;
            }
            catch(ArithmeticException ae){
                JOptionPane.showMessageDialog(this, "divider must not be 0.");
            }


            lbl.setText(String.valueOf(c));
        }
    }
    public static void main(String[] args) {
       Lab l=new Lab();
    }

}
```

5 Input/Output

Output of the program is given below.

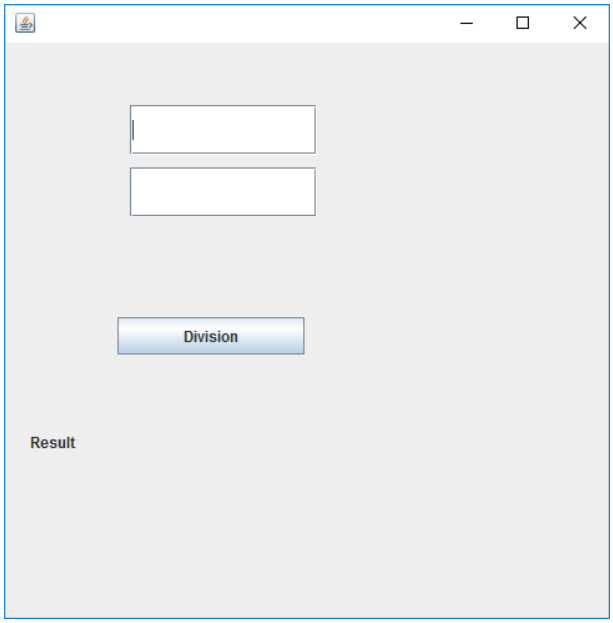

Figure 1: Before providing input.

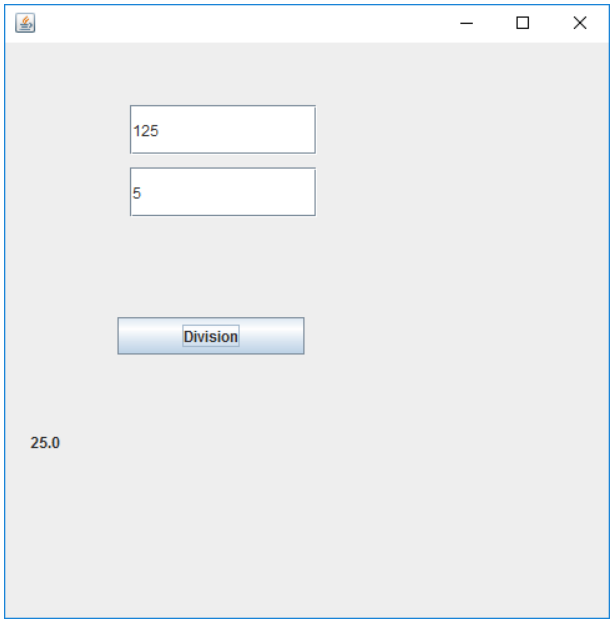

Figure 2: After providing integer inputs and clicking Division button. Here the divider was not 0.

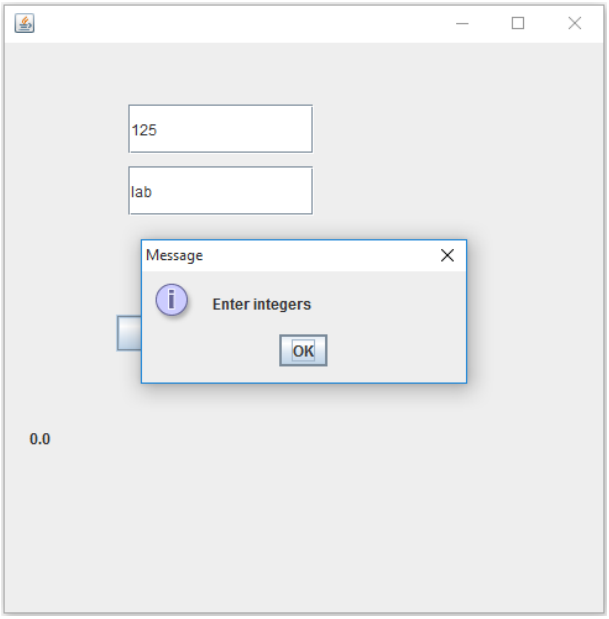

Figure 3: After providing integer inputs and using Division button when an input was not integer.

© Dept. of Computer Science and Engineering, GUB

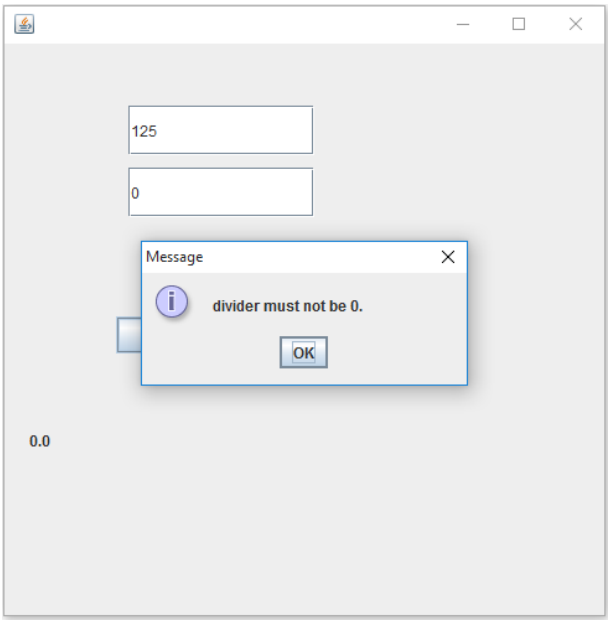

Figure 4: After providing integer input and using Division button. Divider was 0.

6 Discussion

We have used the constructor of the class to set up JFrame and added components to it. We have used 2 text fields for taking user input and a button for invoking the ActionListener to perform event. When the division button is used we calculated division and provided the result using Label. We also checked for exception during event execution and used message dialogue box to show messages.

7 Lab Task (Please implement yourself and show the output to the instructor)

1. Add addition, division and subtraction to the existing code. So that you have a basic calculator having +,-,*,/.

7.1 Problem analysis

- Why are we implementing ActionListener ?
- Understanding to override actionPerformed() method?
- understanding the workflow of event handling using ActionListener.

8 Lab Exercise (Submit as a report)

Some suggested problems:

- Build a swing based application that takes 2 strings, concatenates them, compares them when 2 buttons clicked, compare and concatenate and shows the output as labels.
- Build a swing based application where you can read some data from file and when when clicked a button the application render the data from file and show it on a textfield.

9 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected. Any other policy applied by course teacher may hold applicable.

10 References

https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html

11 References

1. https://www.javatpoint.com/java-actionlistener
2. https://docs.oracle.com/javase/7/docs/api/java/awt/event/package-summary.html

12 Appendix

If need be, course instructor might provide some additional materials.

© Dept. of Computer Science and Engineering, GUB